

ASTR 5820 — Python Setup

Do this before Tuesday of Week 2. Budget thirty minutes. If it takes longer than an hour, stop and contact the instructor.

You are building one toolbox, `astrotools`, over the whole semester. Week 2 adds four functions to it; Week 13 imports what Week 5 wrote. Set it up once, keep it in git, and never copy a constant from one script into another.

The repository is at <https://github.com/phayne/astr5820>. This document is also in it as `SETUP.md`, where the commands can be copied rather than retyped.

1. Install

These instructions use conda. If you already have Anaconda or Miniconda, that works; if not, Miniforge is the smallest option and comes preconfigured for the conda-forge package channel. If you would rather not install conda at all, skip to *Without conda* below — a plain virtual environment works just as well.

From wherever you keep coursework:

```
BASH
git clone https://github.com/phayne/astr5820.git
cd astr5820
conda env create -f environment.yml
conda activate astr5820
pip install -e .
```

That is the whole install, and it takes about a minute.

`environment.yml` lists only Python and pip. Every course package is declared in `pyproject.toml` and installed by `pip install -e .`, which means conda has almost nothing to solve — the step that can otherwise run for tens of minutes finishes in seconds — and there is one dependency list rather than two that drift apart.

`pip install -e .` installs the toolbox in editable mode, so `import astrotools` works from any directory and picks up your edits immediately. It prevents a common failure: an `ImportError` that depends on which folder you happened to be in.

Every time you come back to the course, activate the environment first:

```
BASH
cd /path/to/astr5820
conda activate astr5820
```

Your prompt will show `(astr5820)`. Nearly every "it worked yesterday" problem is a forgotten activation. To leave the environment, run `conda deactivate`.

Without conda

A plain virtual environment works just as well and needs nothing installed beyond Python 3.10 or later:

```
BASH
git clone https://github.com/phayne/ast5820.git
cd astr5820
python3 -m venv .venv
source .venv/bin/activate
pip install -e .
```

On Windows, replace the activation line with `.venv\Scripts\activate`. Activate with `source .venv/bin/activate` rather than `conda activate astr5820`, and leave it with `deactivate`; everything else in this document is the same.

Week 9 only: rebound

The N-body integrator `rebound` is compiled from C source rather than installed from a wheel, so it is kept out of the main install. You will not need it until Week 9:

```
BASH
pip install -e ".[nbody]"
```

On macOS this needs the command line tools (`xcode-select --install`); on Linux, `gcc`. On Windows it depends on your toolchain — some students have installed it successfully under Anaconda and Jupyter, others have not. If it fails there, use **WSL2** (install Ubuntu from the Microsoft Store and follow the instructions above inside it) or **GitHub Codespaces** (open the repository on GitHub → *Code* → *Codespaces* → *Create codespace*, where `.devcontainer/` sets everything up for you). Try it well before Week 9 so there is time to sort out.

2. Verify

```
BASH
python check_setup.py
```

Every line should read `OK`. The script checks your Python version, your packages, that the toolbox imports, that the provided Euler integrator runs, and that matplotlib can write a file.

It finishes by reporting how many Problem Set 1 and 2 tests pass. Nearly all of them **will fail** — that is the assignment. Setup problems appear above the line of equals signs; assignment problems appear below it.

3. What is in here

```
TEXT
astrotools/      the toolbox you are building
constants.py     every number the course uses. Import from here, always.
cloud/collapse.py Set 1      <- you write four functions
nbody/twobody.py  orbits, initial conditions, conserved quantities (provided)
nbody/integrators.py Set 2    <- you write rk4_step
data.py          loaders for the archival datasets
tests/           validation targets for each coding exercise
ps1/, ps2/       starting points for the Set 1 and Set 2 figures
data/            archival data, with provenance in data/README.md
widgets/         the in-class widgets. Open the .html file in a browser;
                 nothing to install, and they work offline.
check_setup.py   this file's companion
```

4. How a coding exercise works

Every week follows the same three steps.

- 1. **Read the test first.** The folder `tests/` contains the validation targets: for each function, the reference value it must reproduce for a stated set of inputs, and the scaling it must obey when those inputs change. Nothing is hidden — read the test as the specification of what you are being asked to write.
- 2. **Write the function until the test passes.**

```
BASH
pytest tests/test_ps1_collapse.py -v
```

Add `-x` to stop at the first failure and `-k jeans` to run one test.

- 3. **Then apply it to observations.** The tests confirm your code is right; they say nothing about whether the physics is. Each set ends by comparing your result against an archival dataset, which is supplied in `data/` and loaded by the functions in `astrotools/data.py` — you do not need to locate or download anything yourself. For Set 1 that dataset is the *Herschel* Gould Belt core catalog, and the questions are whether the Jeans mass predicts the masses of real cores and whether the free-fall time predicts how long they live.

The week runs in that order for a reason. Tuesday you derive the result and commit to a prediction; Thursday the widget shows you whether the prediction held; then you write the function that produced the widget's numbers and turn it on real data. By the time you open `collapse.py` you have already seen what it should do.

5. When the toolbox gains a package

Later problem sets add dependencies — `rebound` in Week 9, and possibly others. When that happens you will be told to run, from the repository root with your environment active:

```
BASH
git pull
pip install -e .
```

`pip install -e .` is safe to run as often as you like: it installs whatever is new and leaves the rest alone. Because the requirement list lives in `pyproject.toml`, this is the same command whether you set up with conda or with a virtual environment, and it never involves a solve.

If a package cannot come from PyPI, `environment.yml` will be updated instead and you will be told to run `conda env update -f environment.yml`. Do not install course packages ad hoc with `conda install` or `pip install <name>` — an environment that no longer matches the repository is one nobody else can help you debug.

6. Handing work in

Submit a single PDF containing your derivations, your figures, and your answers, plus a link to your repository at the commit you are submitting. Figures must state where their data came from.

Troubleshooting

Symptom	Cause and fix
<code>ModuleNotFoundError: No module named 'astrotools'</code>	You skipped <code>pip install -e .</code> , or the environment is not active. Run <code>conda activate astr5820</code> (or <code>source .venv/bin/activate</code>) from the repository root, then run the install command again.
<code>ModuleNotFoundError</code> for numpy or scipy after installing them	Two Pythons. <code>python check_setup.py</code> prints the interpreter path — confirm it sits inside the <code>astr5820</code> environment, or inside <code>.venv</code> in the repository.
<code>pip install -e .</code> fails to build a package	Almost always means pip could not find a wheel and fell back to compiling. Check that <code>python3 --version</code> is 3.10 or later and that <code>pip install --verbose <pkg></code> has been run — an old pip does not recognize newer wheel tags.

	Or later and that <code>pip install --upgrade pip</code> has been run — an old pip does not recognize newer wheel tags.
conda's <code>Solving environment</code> runs for more than a minute	You are using an older copy of <code>environment.yml</code> . Run <code>git pull</code> — the current file lists only Python and pip, which solves almost instantly. Ctrl-C will not interrupt a solve in progress, since it runs inside a C extension that ignores the signal; open a second terminal, run <code>kill -f conda</code> , then <code>conda env remove -n astr5820</code> before retrying.
<code>rebound</code> fails to install	It compiles from source and needs a C compiler: <code>xcode-select --install</code> on macOS, <code>gcc</code> on Linux. On Windows, use WSL2 or Codespaces (§1). Not needed before Week 9.
Figures do not appear when running a script	Scripts save to <code>figures/</code> rather than opening a window. Open the file. In Jupyter, add <code>%matplotlib inline</code> .
A test fails by roughly a factor of 1.5	Almost always μ : mass density is <code>n(H2) * mu * m_H</code> , and <code>sqrt(2.33) = 1.53</code> .
A test fails by orders of magnitude	Units. Everything in this course is SI, including inputs. Convert at the boundary, never inside a formula.
Jupyter cannot see the environment	With the environment active, run <code>pip install -e ".[notebook]"</code> then <code>python -m ipykernel install --user --name astr5820</code> , and pick that kernel.

Still stuck after fifteen minutes? Contact the instructor, including the full output of `python check_setup.py`.